



Introduction to Computer Vision

From Pixels to Perception — and Your First YOLO Model

What is Computer Vision?



A field of AI that enables machines to interpret and understand the visual world — images and video — the way humans do.



Analyze Images

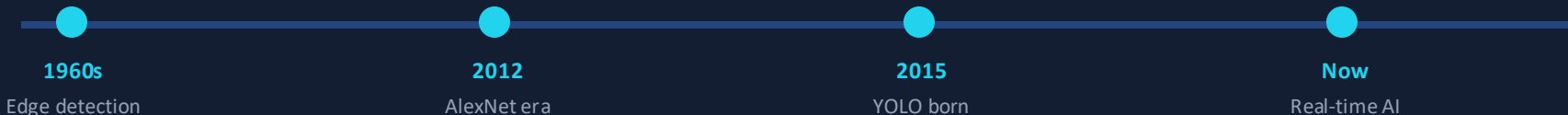


Detect Objects



Make Decisions

CV sits at the intersection of AI, deep learning, and image processing.



Real-World Applications



Self-Driving Cars

Lane detection, pedestrian recognition, obstacle avoidance



Medical Imaging

Tumor detection, X-ray analysis, retinal scans



Manufacturing

Defect inspection, quality control, robotic guidance



Security & Surveillance

Face recognition, motion detection, anomaly alerts



Agriculture

Crop health monitoring, weed detection, yield prediction



Retail & E-Commerce

Visual search, AR try-on, shelf analytics

CV is already a \$20B+ market and growing rapidly across every industry.

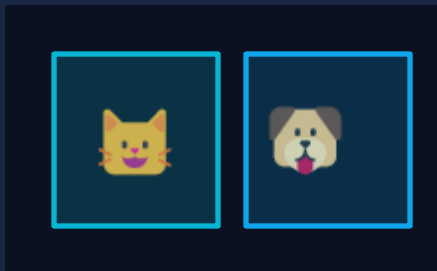
Core Computer Vision Tasks



Image Classification

Assign a label to the entire image.

"This is a cat."



Object Detection

Locate & classify multiple objects with bounding boxes.

"Cat at (x,y), Dog at (x,y)."



Segmentation

Pixel-level classification — outline exact shape of each object.

Object Detection is the focus of this class — and YOLO is the king of real-time detection.

How Do Machines "See"?

Pixels → Numbers → Patterns

120	145	180	210	235
80	120	145	180	210
30	80	120	145	180

Each pixel = a number (0-255)

Images are 3D arrays: Height × Width × Channels (RGB)

Feature Extraction Pipeline



Raw Pixels



Edges & Textures

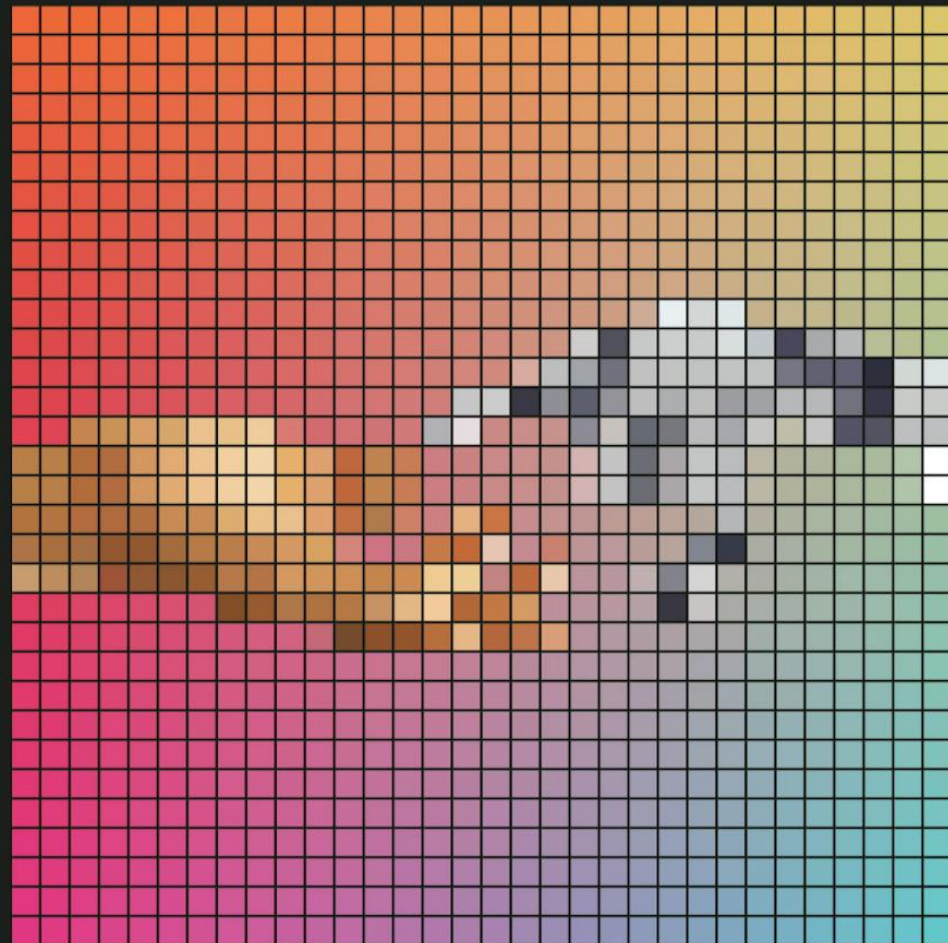


Shapes & Parts

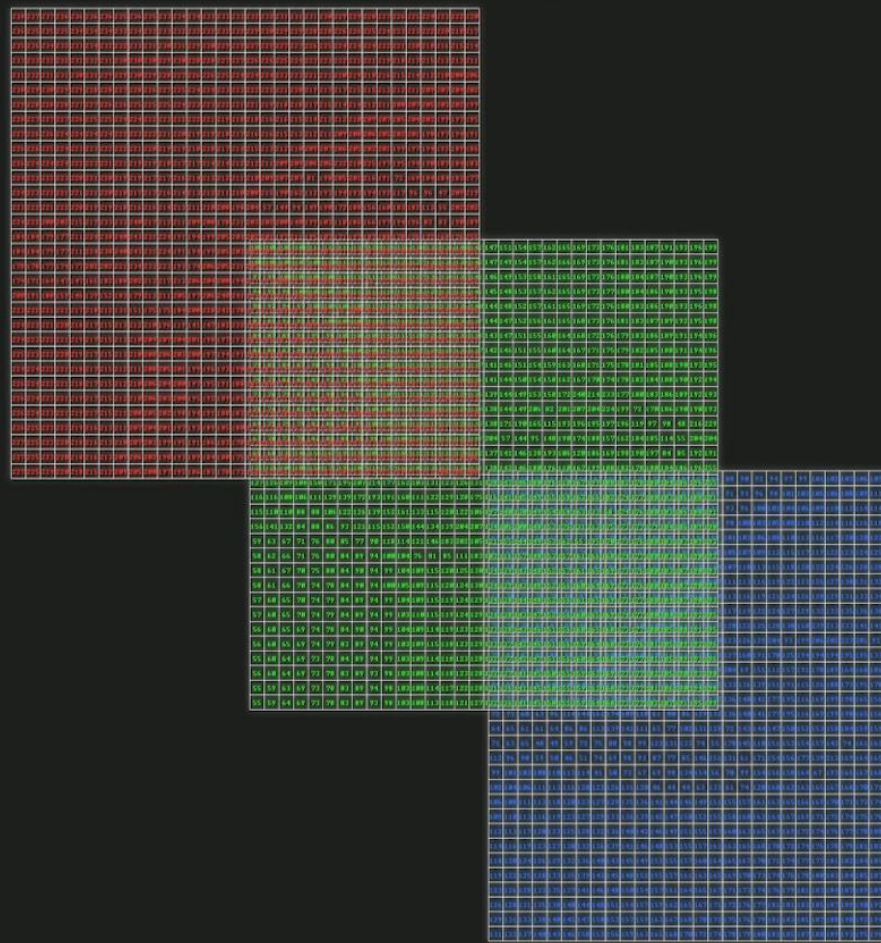


Object Recognition

INPUT



CHANNEL





2	2	4	2	2
1	1	2	1	1
0	0	0	0	0
-1	-1	-2	-1	-1
-2	-2	-4	-2	-2



1	1	0	-1	-1
1	1	0	-1	-1
0	0	0	0	0
-1	-1	0	1	1
-1	-1	0	1	1

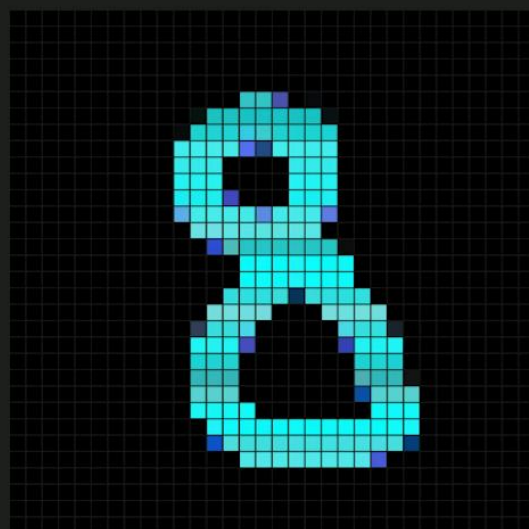
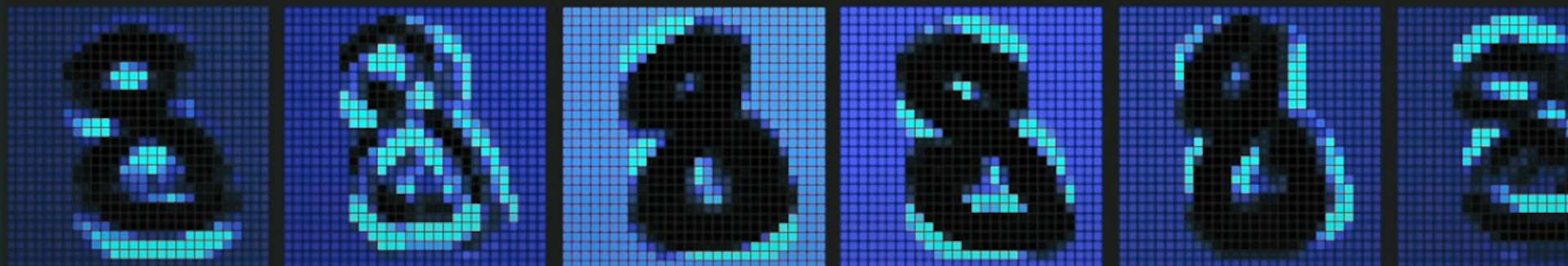


0	1	2	3	4
-1	0	1	2	3
-2	-1	0	1	2
-3	-2	-1	0	1
-4	-3	-2	-1	0

0	0	-1	0	0
0	0	-1	0	0
-1	-1	5	-1	-1
0	0	-1	0	0
0	0	-1	0	0



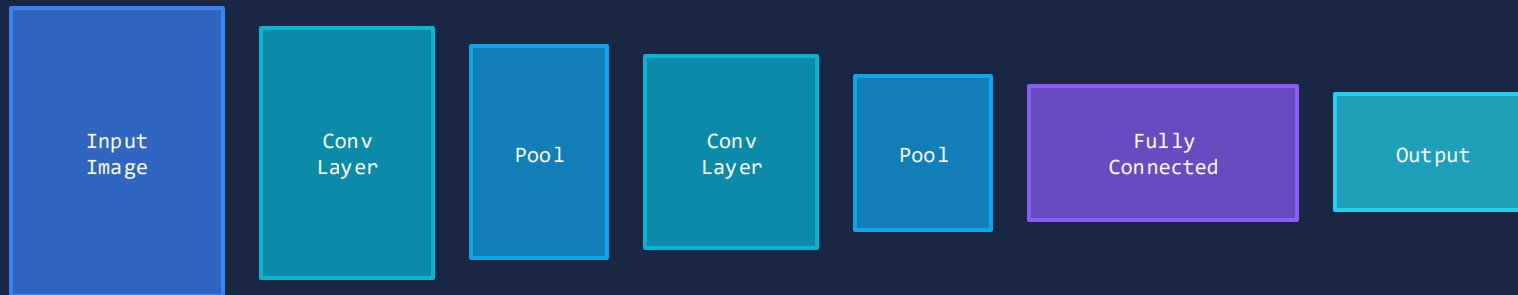
Convolutional



Input

Deep Learning & CNNs

Convolutional Neural Network (CNN) Architecture



Convolution

Slides a filter/kernel across the image to detect edges, textures, and patterns.



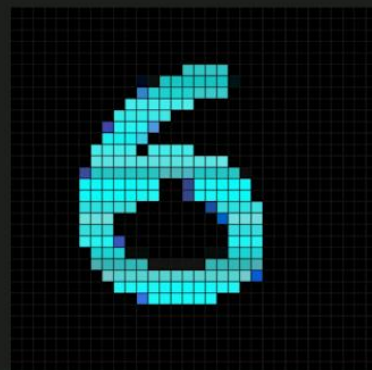
Pooling

Reduces spatial size, keeping the most important features. Makes model faster.



Fully Connected

Final layers that combine all learned features to make a classification decision.



Input
32x32
1024 Pixels

Conv1
6x28x28
4704 Pixels

Pool1
6x14x14
1176 Pixels

Conv2
16x10x10
1600 Pixels

Pool2
16x5x5
400 Pixels

What is YOLO?

Y

You

O

Only

L

Look

O

Once



The Key Innovation

Unlike older methods that scan an image in parts, YOLO processes the entire image in a single pass through the neural network. This makes it incredibly fast — fast enough for real-time video.

45-155+
FPS

Real-time speed
on modern GPUs

2015

Created by Joseph
Redmon et al.

How YOLO Works

1



Divide

The image is divided into an $S \times S$ grid of cells (e.g. 7×7 or 13×13).



2



Predict

Each grid cell predicts B bounding boxes, confidence scores, and class probabilities simultaneously.



3



Filter

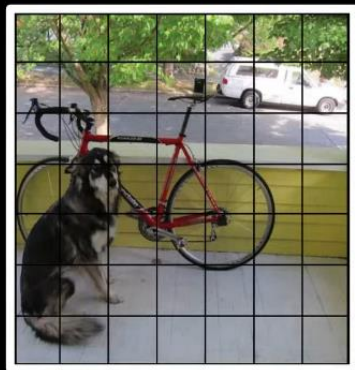
Non-Max Suppression (NMS) removes overlapping boxes, keeping only the best detections.

Each prediction = $[x, y, w, h, \text{confidence}, \text{class}_1, \text{class}_2, \dots, \text{class}_n]$

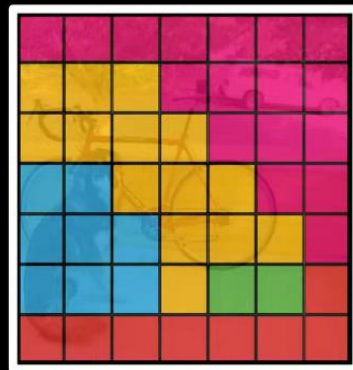
The YOLO Algorithm



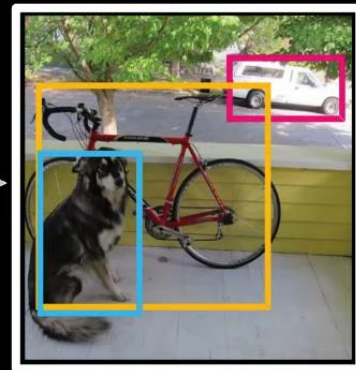
Input



$S \times S$ grid



Class probability map



Detections

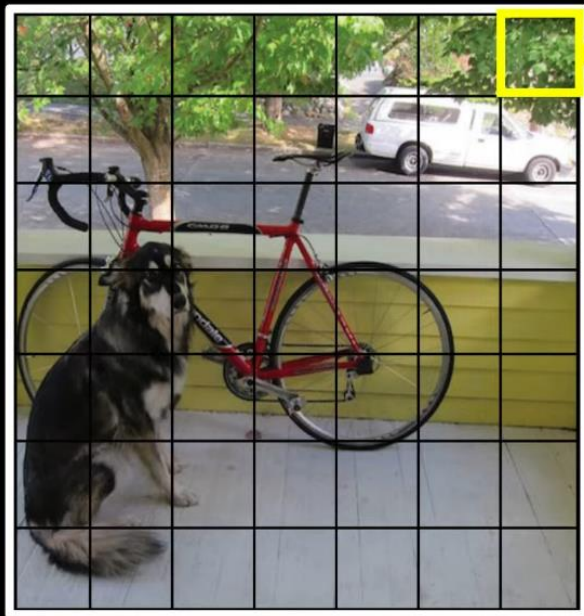
Redmon *et. al* (2016)

"You Only Look Once: Unified, Real-Time Detection"

- | | | | |
|--------|---------|--------|--------------|
| Yellow | Bicycle | Green | Desk |
| Pink | Car | Orange | Dining table |
| Blue | Dog | ... | |

The YOLO Algorithm $(B \times 5 + n)$

Output vector length



$S \times S$ grid

$[p(c_1), p(c_2), \dots, p(c_n)]$
Class probabilities

$[x_1, y_1, \sqrt{w_1}, \sqrt{h_1}, C_1]$

$[x_2, y_2, \sqrt{w_2}, \sqrt{h_2}, C_2]$

...

$[x_B, y_B, \sqrt{w_B}, \sqrt{h_B}, C_B]$

Bounding box predictions

YOLO Evolution



Today, YOLOv8/v11 from Ultralytics is the go-to for most real-time detection tasks.

YOLO vs. Other Detectors

	YOLO	Faster R-CNN	SSD
Speed	⚡ Very Fast (45+ FPS)	🐢 Slow (5-7 FPS)	🏃 Fast (20+ FPS)
Accuracy	High (mAP ~50-55)	Very High (mAP ~55+)	Moderate (mAP ~46)
Architecture	Single-stage	Two-stage	Single-stage
Best For	Real-time apps	High-precision tasks	Embedded devices

YOLO's single-stage approach offers the best speed-to-accuracy tradeoff for most practical applications.

Hands-On: Build with Roboflow

1



Upload Images

Drag & drop your dataset into Roboflow. Supports JPG, PNG, video frames.

2



Annotate

Draw bounding boxes around objects. Use Roboflow's smart labeling tools.

3



Augment & Preprocess

Auto-resize, flip, rotate, adjust brightness. More data = better model.

4



Train

One-click training on Roboflow's cloud. Pick YOLOv8 or other architectures.

5



Deploy

Get an API endpoint, edge deploy, or export weights for local inference.

Is Your Dataset Ready?

✓ LOOK FOR THIS

- ✓ 200+ images minimum (500+ is better)
- ✓ Labeled / annotated images already
- ✓ Variety of angles, lighting & backgrounds
- ✓ Clear class names (e.g. 'cat', 'dog', not '1', '2')
- ✓ Balanced classes — roughly equal per label

✗ AVOID THIS

- ✗ Fewer than 50 images per class
- ✗ No labels or annotations provided
- ✗ All images from the same angle/scene
- ✗ Blurry, low-resolution, or compressed images
- ✗ Classes that look too similar to each other

Let's Build Something!



Computer Vision teaches machines to see and understand images



YOLO enables real-time object detection in a single pass



CNNs learn features from raw pixels — no hand-crafting needed



Roboflow makes it easy to go from dataset to deployed model

Next Up: Open Roboflow → Upload a dataset → Train your first YOLO model!